

Contents

1 数学	1
1.1 组合数学	1
1.1.1 组合数预处理	1
1.1.2 斯特林数	2
1.2 筛法	3
1.3 费马小定理	7
1.4 欧拉函数	7
1.5 欧拉定理	7
1.5.1 扩展欧拉定理	7
1.6 威尔逊定理	7
1.6.1 推论:	7
1.7 裴蜀定理 (贝祖定理)	7
1.7.1 推广:	7
1.8 扩展欧几里得算法	8
1.8.1 exgcd 求逆元 (需满足 $\gcd(a, \text{mod}) = 1$):	8
1.9 中国剩余定理	9
1.9.1 扩展中国剩余定理	9
1.10 BSGS 算法	11
1.11 矩阵	12
1.12 矩阵快速幂	16
1.13 哥德巴赫猜想	17
1.14 整除分块	18

1 数学

1.1 组合数学

1.1.1 组合数预处理

1.1.1.1 组合数预处理 记得 init, 如果模数不一样也要变, 其中 $f[i]$ 表示 i 的阶乘, $g[i]$ 表示 i 阶乘的逆元

```
const int mod = 998244353;
long long qpow(long long a, long long n){
    a %= mod;
    long long res = 1;
    while(n){
        if(n&1) res *= a, res %= mod;
        a *= a, a %= mod;
        n >>= 1;
    }
    return res;
}

vector<int> f, g;
void init(int size){
    f.resize(size+1); g.resize(size+1);
    f[0] = g[0] = 1;
    for(int i = 1; i <= size; i++){
        f[i] = 1LL*f[i-1]*i%mod;
        g[i] = 1LL*g[i-1]*qpow(i, mod-2)%mod;
    }
}

long long C(int a, int b){
    if(b > a || b < 0) return 0;
    return 1LL*f[a]*g[b]%mod*g[a-b]%mod;
}
```

```

long long A(int a,int b){
    return 1LL*f[a]*g[a-b]%mod;
}
long long lucas(int a,int b){
    if(b>a || b<0) return 0;
    if(a == 0 || b == 0) return 1;
    return lucas(a/mod,b/mod)*C(a%mod,b%mod)%mod;
}

```

1.1.1.2 枚举固定大小的子集 gopersHack

```

auto nxt = [&](int x)->int {
    // 生成下一个更大的同样二进制有 k 个 1 的数
    // int 注意越界
    int c = x & -x;
    int r = x + c;
    return ((r ^ x) >> 2) / c | r;
};
// 例如一共有 10 位, 枚举 1 的个数为 4 的所有数字
for(int i = (1<<4)-1; i<(1<<10); i = nxt(i)){
    cout<<i<<" ";
}
cout<<"\n";

```

1.1.2 斯特林数

1.1.2.1 第一类斯特林数 斯特林轮换数 实际意义: 把 n 个不同的元素, 划分为 m 个非空圆排列的方案数

递推公式

$$[n, m] = [n-1, m-1] + (n-1) * [n-1, m]$$

从组合意义上讲即自己作为一个新的轮换 或者自己插入到已有的人的左边

边界 $[0, 0] = 1$

```

int S[n+1][m+1];
S[0][0] = 1;
for(int i = 1; i<=n; i++){
    for(int j = 1; j<=m; j++){
        S[i][j] = (S[i-1][j-1]+1LL*(i-1)*S[i-1][j])%mod;
    }
}

```

1.1.2.2 第二类斯特林数 斯特林子集数 实际意义: 把 n 个不同的元素, 划分为 m 个非空子集的方案数

递推公式

$$\{n, m\} = \{n-1, m-1\} + m * \{n-1, m\}$$

从组合意义上讲即自己作为一个新的子集 或者自己插入已有的子集中

常用公式

$$x^n = \sum_{k=0}^n \{n, k\} (x, k) k!$$

常用化简 $n \wedge m$

递推边界 $\{0, 0\} = 1$

计算公式

$$\{n, m\} = \sum_{i=0}^m \frac{(-1)^{m-i} i^n}{i!(m-i)!}$$

1.2 筛法

- 质数筛:

```
std::vector<int> minp, primes;

void sieve(int n) {
    minp.assign(n + 1, 0); primes.clear();

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            primes.push_back(i);
        }

        for (auto p : primes) {
            if (i * p > n) break;
            minp[i * p] = p;

            if (p == minp[i]) break;
        }
    }
}
```

- 区间素数筛 P1835

获取区间 $[l, r]$ 中的所有素数, 复杂度 $O(\sqrt{r} + (r - l) * \log\log(r))$

```
std::vector<int> prime;

void Interval(int l, int r) {
    sieve(1e5);
    if (l == 1) l++;
    std::vector<int> a(r - l + 1);
    for (i64 p: primes) {
        i64 st = std::max(2ll, (l - 1) / p + 1) * p;
        for (i64 i = st; i <= r; i += p) {
            if (i - l >= 0) a[i - l] = 1;
        }
    }
    for (int i = 0; i <= r - l; i++) {
        if (!a[i]) prime.push_back(i + l);
    }
}
```

- 欧拉函数 (试除法)

```
int phi(int x){
    int ans = x;
    for(int i=2;i*i<=x;i++){
        if(x%i==0){
            while(x%i==0) x/=i;
            ans=ans*(i-1)/i;
        }
    }
    if(x>1) ans=ans*(x-1)/x;
    return ans;
}
```

- 欧拉函数 (筛法)

```

std::vector<i64> minp, phi, primes;

void Phi(int n) {
    minp.assign(n + 1, -1), phi.resize(n + 1), phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (minp[i] == -1) {
            minp[i] = i, phi[i] = i - 1;
            primes.push_back(i);
        }
        for (auto &p: primes) {
            if (i * p > n) break;
            minp[i * p] = p;
            if (minp[i] == p) {
                phi[i * p] = phi[i] * p;
                break;
            } else {
                phi[i * p] = (p - 1) * phi[i];
            }
        }
    }
}

```

- 筛法求每个数 **不同质因子的个数**

fac[i] 表示数字 i 的**不同质因子个数**

```

std::vector<int> minp, fac, primes;

void sieve(int n) {
    minp.resize(n + 1);
    fac.resize(n + 1);
    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            fac[i] = 1;
            primes.push_back(i);
        }
        for (int p: primes) {
            if (i * p > n) break;
            minp[i * p] = p;
            if (i % p == 0) {
                fac[i * p] = fac[i];
                break;
            } else {
                fac[i * p] = fac[i] + 1;
            }
        }
    }
}

```

- 筛法求有几个质因子

fac[i] 表示数字 i 的**所有质因子个数** (包含相同质因子)

$$fac[i] = d(n) = (a_1 + 1)(a_2 + 1) \dots (a_k + 1)$$

```

std::vector<int> minp, fac, primes;
void prime_fac(int n) {
    minp.resize(n+1); fac.resize(n+1);
    for(int i=2; i<=n; i++){

```

```

        if(minp[i]==0){
            minp[i]=i;
            fac[i]=1;
            primes.push_back(i);
        }
        for(int p:primes){
            if(i*p>n) break;
            minp[i*p]=p;
            fac[i*p]=fac[i]+1;
            if(i%p==0) break;
        }
    }
}

```

- 筛法求因子个数

fac[i] 表示数字 i 的**所有因子个数**

mi[i] 表示数字 i 的**最小质因子的指数**

```

std::vector<int> primes,minp,mi,fac;
void factor(int n){
    minp.resize(n+1),mi.resize(n+1),fac.resize(n+1);
    fac[1]=1;
    for(int i=2;i<=n;i++){
        if(!minp[i]){
            primes.push_back(i);
            mi[i]=1,fac[i]=2;
        }
        for(int &p:primes){
            if(p*i>n) break;
            minp[i*p]=p;
            if(i%p==0){
                mi[i*p]=mi[i]+1;
                fac[i*p]=fac[i]/mi[i*p]*(mi[i*p]+1);
                break;
            }else{
                mi[i*p]=1,fac[i*p]=fac[i]*2;
            }
        }
    }
}

```

- 筛法求因数和

$$mis[i] = 1 + p_1^1 + p_1^2 + \dots + p_1^{a_1} = \frac{p_1^{a_1+1}-1}{p_1-1}$$

$$facs[i] = \sigma(i) = (1 + p_1 + \dots + p_1^{a_1}) \times (1 + p_2 + \dots + p_2^{a_2}) \times \dots$$

```
std::vector<int> primes,minp,mis,facs;
```

```

void factors(int n){
    minp.resize(n+1),mis.resize(n+1),facs.resize(n+1);
    for(int i=2;i<=n;i++){
        if(minp[i]==0){
            primes.push_back(i);
            mis[i]=facs[i]=i+1;
        }
        for(int &p:primes){
            if(i*p>n) break;

```

```

        minp[i*p]=p;
        if(i%p==0){
            mis[i*p]=mis[i]*p+1;
            facs[i*p]=facs[i]/mis[i]*mis[i*p];
            break;
        }else{
            mis[i*p]=p+1;
            facs[i*p]=facs[i]*mis[i*p];
        }
    }
}
}
}

```

- 筛法求莫比乌斯函数

```

std::vector<int> minp, primes, mu;

void Mobius(int n) {
    mu.resize(n + 1), minp.resize(n + 1);
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!minp[i]) {
            primes.push_back(i);
            minp[i] = i, mu[i] = -1;
        }
        for (int &p: primes) {
            if (i * p > n) break;
            minp[i * p] = p;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = -mu[i];
            }
        }
    }
}
}
}

```

1.3 费马小定理

若 p 为质数, 且 a, p 互质, 则 $a^{p-1} \equiv 1 \pmod{p}$

$$a * a^{p-2} = a^{p-1} \equiv 1 \pmod{p}$$

所以快速幂求 a 模意义下的逆元就是 $\text{qpow}(a, \text{mod}-2)$ 只适用于 p 为质数

1.4 欧拉函数

定义: $1 \sim n$ 中与 n 互质的数的个数, 记为 $\phi(n) = \prod_{i=1}^s \frac{p_i-1}{p_i}$

```
int get_phi(int x) {
    int res = x;
    for (int i = 2; i * i <= x; i++) {
        if (x % i == 0) {
            while (x % i == 0) { x /= i; }
            res = res / i * (i - 1);
        }
    }
    if (x > 1) res = res / x * (x - 1);
    return res;
}
```

1.5 欧拉定理

若 $\gcd(a, m) = 1$, 则 $a^{\phi(m)} \equiv 1 \pmod{m}$

1.5.1 扩展欧拉定理

不需要满足 $\gcd(a, m) = 1$

$$a^b \equiv \begin{cases} a^b, & b < \phi(m) \\ a^{b \pmod{\phi(m)} + \phi(m)}, & b \geq \phi(m) \end{cases}$$

当 $\gcd(a, m) \neq 1$ 时, 想象一个圈, 外面漏了一个尾巴的结构, 起初 a 位置尾巴的位置, 而圈表示周期长度是 $\phi(m)$, 从 a 进入圈需要的步数不超过 $\phi(m)$, $b < \phi(m)$ 的时候直接快速幂计算, 因为终点可能还在尾巴上。否则 a 最后的位置一定落在圈内, 前走 $\phi(m)$ 步进入圈, 然后走余数步就可以了

1.6 威尔逊定理

$(p-1)! \equiv -1 \pmod{p}$ 是 p 为质数的充分必要条件

1.6.1 推论:

- 若 p 是质数, 则 $(p-1)! + 1 \equiv 0 \pmod{p}$
- 若 p 是大于 4 的合数, 则 $(p-1)! \equiv 0 \pmod{p}$

1.7 裴蜀定理 (贝祖定理)

一定存在整数 x, y 满足 $ax + by = \gcd(a, b)$

1.7.1 推广:

- 一定存在整数 x, y 满足 $ax + by = \gcd(a, b) * n$
- 一定存在整数 $X_1 \dots X_n$ 满足 $\sum_{i=1}^n A_i X_i = \gcd(A_1, A_2, \dots, A_n)$

1.8 扩展欧几里得算法

求 $ax+by=\gcd(a,b)$ 的特解 (x_0, y_0)

//扩展欧几里得算法

//求出 $a*x+b*y=\gcd(a,b)$ 式子中, 满足条件的 x 与 y , 同时求出 $\gcd(a,b)$;

// x, y 为经过运算最初得到的数

//先求特解 $(x_0, y_0) \rightarrow x_0 = x * c / \gcd, y_0 = y * c / \gcd$;

//再构造通解 $X = x_0 + (b * k) / \gcd(a, b), Y = y_0 - (a * k) / \gcd(a, b)$

//最小正整数解 $X_{min} = (x_0 \% (b/g) + (b/g)) \% (b/g), Y_{min} = (y_0 \% (a/g) + (a/g)) \% (a/g)$

```
i64 exgcd(i64 a, i64 b, i64 &x, i64 &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    i64 gcd = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return gcd;
}
```

1.8.1 exgcd 求逆元 (需满足 $\gcd(a, \text{mod}) = 1$):

```
i64 getInv(i64 a, i64 mod) {
    i64 x, y;
    i64 d = exgcd(a, mod, x, y);
    if (d != 1) return -1;
    return (x % mod + mod) % mod;
}
```

1.9 中国剩余定理

求解（模数两两互质的）线性同余方程：

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \dots \\ x \equiv r_n \pmod{m_n} \end{cases}$$

```
template<class T>
T exgcd(T a, T b, __int128 &x, __int128 &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    T gcd = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return gcd;
}

template<class T>
T CRT(const std::vector<T> &m, const std::vector<T> &r) {
    T M = 1, ans = 0;
    for (int i = 0; i < m.size(); i++) M *= m[i];
    for (int i = 0; i < m.size(); i++) {
        T c = M / m[i];
        __int128 x, y;
        exgcd(c, m[i], x, y);
        ans = (ans + r[i] * c * x % M) % M;
    }
    return (ans % M + M) % M;
}
```

1.9.1 扩展中国剩余定理

求解（模数两两不一定互质的）线性同余方程

```
template<class T>
T exgcd(T a, T b, __int128 &x, __int128 &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    T gcd = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return gcd;
}

template<class T>
T EXCRT(const std::vector<T> &m, const std::vector<T> &r) { //有  $x \equiv r_1 \pmod{m_1}, x \equiv r_2 \pmod{m_2}$ 
    //转化为  $x = m_1 * p + r_1 = m_2 * q + r_2 \quad (1)$ 
    __int128 m1, m2, r1, r2, p, q;
    //  $m_1 * p - m_2 * q = r_2 - r_1$ 
    m1 = m[0], r1 = r[0];
    int n = m.size();
    for (int i = 1; i < n; i++) {
        m2 = m[i], r2 = r[i];
        __int128 d = exgcd(m1, m2, p, q);
```

```

//由裴蜀定理--> 当  $\gcd(m1, m2) | (r2 - r1)$  时 有解
if ((r2 - r1) % d) { return -1; }
//由扩欧算法得 特解  $p = p * (r2 - r1) / \gcd, q = q * (r2 - r1) / \gcd$ 
p = p * (r2 - r1) / d; //特解
//p 可能是负数, 为了方便要使 p 变成正数, 通解--》  $P = p + m2 / \gcd * k, Q = q - m1 / \gcd * k$ 
p = (p % (m2 / d) + m2 / d) % (m2 / d);
// $x = m1 * P + r1 = (m1 * m2) / \gcd * k + m1 * p1 + r1$ , 与 (1) 比对
r1 = m1 * p + r1;
//得  $r = m1 * p + r1, m = \text{lcm}(m1, m2)$ 
m1 = m1 * m2 / d;
}
return (r1 % m1 + m1) % m1;
}

```

1.10 BSGS 算法

求解高次同余方程

给定整数 a, b, p , a, p 互质 (a, p 不互质时用 `exbsgs`), 求满足 $a^x \equiv b \pmod{p}$ 的最小非负整数 x

`bsgs` 本身复杂度是 $\log(p)$ 的, 这个板子用的 `map`, 复杂度带个 $\log$, `tle` 的话换 `umap` 试试

```
//a^x=b(mod p), 求最小的非负整数 x (a,p 互质)
//根据扩展欧拉定理 a^x=a^(x%phi(p)) (mod p), 可知 a^x 模 p 意义下的最小循环节为 phi(p)
//令 x=im-j, 其中 m=ceil(sqrt(p)), i~(1--m), j~(0,m-1)
template<class T>
struct BSGS {
    T bsgs(T a, T b, T p) {
        a %= p, b %= p;
        if (b == 1) return 0;
        T m = ceil(sqrt((double) p)), t = b;
        std::map<T, int> doc; doc[b] = 0;
        for (int i = 1; i < m; i++) {
            t = t * a % p;
            doc[t] = i;
        }
        T mi = 1; t = 1;
        for (int i = 1; i <= m; i++) mi = mi * a % p;

        for (int i = 1; i <= m; i++) {
            t = t * mi % p;
            if (doc.count(t)) return i * m - doc[t];
        }
        return -1;
    }
};

T exbsgs(T a, T b, T p) {
    a %= p, b %= p;
    if (b == 1 || p == 1) return 0;
    T k = 0, A = 1;
    while (true) { //使 a,p 互质
        T g = __gcd(a, p);
        if (g == 1) break;
        if (b % g != 0) return -1;
        k++, b /= g, p /= g;
        A = A * (a / g) % p;
        if (A == b) return k;
    }
    T m = ceil(sqrt((double) p)), t = b;
    std::map<T, int> doc; doc[t] = 0;
    for (int i = 1; i < m; i++) {
        t = t * a % p;
        doc[t] = i;
    }
    T mi = 1; t = A;
    for (int i = 1; i <= m; i++) mi = mi * a % p;
    for (int i = 1; i <= m; i++) {
        t = t * mi % p;
        if (doc.count(t)) return i * m - doc[t] + k;
    }
    return -1;
};
```

1.11 矩阵

```
using i64 = long long;
template<class T, int R, int C>
struct Matrix {
    std::array<T, R * C> A;
    int n{}, m{};

    Matrix(int _r = R, int _c = C) {
        init(_r, _c);
    }

    void init(int _r, int _c) {
        n = _r, m = _c;
        A.fill(0);
    }

    int id(int i, int j) const {
        return i * m + j;
    }

    T at(int i, int j) const {
        assert(id(i, j) >= 0 && id(i, j) < n * m);
        return A[id(i, j)];
    }

    T &operator[](int i) {
        assert(i >= 0 && i < n * m);
        return A[i];
    }

    Matrix<T, R, C> &operator=(const std::array<T, R * C> &a) {
        A = a;
        return *this;
    }

    template<int N, int M>
    Matrix<T, R, M> operator*(const Matrix<T, N, M> &rhs) {
        assert(C == N);
        Matrix<T, R, M> c(n, rhs.m);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < rhs.m; j++) {
                for (int k = 0; k < m; k++) {
                    c[c.id(i, j)] = c.at(i, j) + A[id(i, k)] * rhs.at(k, j);
                }
            }
        }
        return c;
    }

    Matrix<T, R, C> operator*(const T &a) {
        Matrix<T, R, C> c(n, m);
        for (int i = 0; i < n * m; i++) {
            c[i] = a * A[i];
        }
        return c;
    }
}
```

```

friend Matrix<T, R, C> operator*(const T &a, const Matrix<T, R, C> &lhs) {
    Matrix<T, R, C> c(lhs.n, lhs.m);
    for (int i = 0; i < lhs.n * lhs.m; i++) {
        c[i] = a * lhs[i];
    }
    return c;
}

Matrix<T, R, C> operator+(const Matrix<T, R, C> &rhs) {
    Matrix<T, R, C> c(n, m);
    for (int i = 0; i < n * m; i++) {
        c[i] = A[i] + rhs.A[i];
    }
    return c;
}

Matrix<T, R, C> operator-(const Matrix<T, R, C> &rhs) {
    Matrix<T, R, C> c(n, m);
    for (int i = 0; i < n * m; i++) {
        c[i] = A[i] - rhs.A[i];
    }
    return c;
}

friend bool operator==(const Matrix<T, R, C> &lhs, const Matrix<T, R, C> &rhs) {
    if (lhs.n != rhs.n && lhs.m != rhs.m) return false;
    for (int i = 0; i < lhs.n * lhs.m; i++) {
        if (lhs[i] != rhs[i]) return false;
    }
    return true;
}

friend bool operator!=(const Matrix<T, R, C> &lhs, const Matrix<T, R, C> &rhs) {
    if (lhs.n != rhs.n && lhs.m != rhs.m) return true;
    for (int i = 0; i < lhs.n * lhs.m; i++) {
        if (lhs[i] != rhs[i]) return true;
    }
    return false;
}

friend constexpr std::istream &operator>>(std::istream &is, Matrix<T, R, C> &rhs) {
    for (int i = 0; i < rhs.n * rhs.m; i++) {
        is >> rhs[i];
    }
    return is;
}

friend constexpr std::ostream &operator<<(std::ostream &os, Matrix<T, R, C> &rhs) {
    for (int i = 0; i < rhs.n; i++) {
        for (int j = 0; j < rhs.m; j++) {
            os << rhs.at(i, j) << " ";
        }
        os << " \n"[i < rhs.n - 1];
    }
    return os;
}
};

```

```

template<class T, int R>
Matrix<T, R, R> unit(int N) { //单位矩阵
    Matrix<T, R, R> E(N, N);
    for (int i = 0; i < N; i++) {
        E[E.id(i, i)] = 1;
    }
    return E;
}

template<class T, int R, int C>
Matrix<T, C, R> transpose(const Matrix<T, R, C> &A) { //矩阵转置
    Matrix<T, C, R> B(A.m, A.n);
    for (int i = 0; i < A.n; i++) {
        for (int j = 0; j < A.m; j++) {
            B[B.id(j, i)] = A.at(i, j);
        }
    }
    return B;
}

template<class T, int R>
T det(const Matrix<T, R, R> &A) { //方阵的行列式
    int n = A.n, m = A.m;
    assert(n == m);
    Matrix<long double, R, R> B(n, m);
    for (int i = 0; i < n * m; i++) B[i] += A.at(0, i);
    for (int c = 0; c < m; c++) {
        int ok = false, r = c;
        for (r = c; r < n; r++) {
            if (B.at(r, c) != 0) {
                ok = true;
                for (int j = 0; j < n; j++) {
                    std::swap(B[B.id(c, j)], B[B.id(r, j)]);
                }
                break;
            }
        }
        if (!ok) return 0;
        r++;
        for (; r < n; r++) {
            if (B.at(r, c) == 0) continue;
            long double t = B.at(r, c) / B.at(c, c);
            for (int j = c; j < n; j++) {
                B[B.id(r, j)] -= B[B.id(c, j)] * t;
            }
        }
    }
    T ans = 1;
    for (int i = 0; i < n; i++) {
        ans *= B.at(i, i);
    }
    return ans;
}

```

```

template<class T, int R, int C>
Matrix<T, R, C> adjont(const Matrix<T, R, C> &A) { //伴随矩阵
    int n = A.n, m = A.m;
    assert(n == m);
    Matrix<T, R, C> B(n, n);
    Matrix<T, R - 1, R - 1> tmp(n - 1, n - 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int idx = 0;
            for (int x = 0; x < n; x++) {
                for (int y = 0; y < n; y++) {
                    if (x == i || y == j) continue;
                    tmp[idx++] = A.at(x, y);
                }
            }
            B[B.id(j, i)] = ((i + j) % 2 ? -1 : 1) * det(tmp);
        }
    }
    return B;
}

```

```

template<class T, int R, int C>
Matrix<T, R, C> power(Matrix<T, R, C> a, i64 b) {
    auto ans = unit<T, R>(a.n);
    assert(a.n == a.m);
    for (; b; b >>= 1, a = a * a) {
        if (b & 1) ans = ans * a;
    }
    return ans;
}

```

1.12 矩阵快速幂

```
using i64 = long long;

constexpr int N = 100;
constexpr int mod = 1e9 + 7;
int M = N;
using Matrix = std::array<i64, N * N>;

int id(int i, int j) {
    return i * M + j;
}

Matrix init(Matrix &mat, int x = 0) {
    if (x == 0) {
        mat.fill(0);
    } else {
        for (int i = 0; i < M; i++) mat[id(i, i)] = x;
    }
    return mat;
}

Matrix operator*(const Matrix &a, const Matrix &b) {
    Matrix c{};
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < M; j++) {
            for (int k = 0; k < M; k++) {
                c[id(i, j)] = (c[id(i, j)] + a[id(i, k)] * b[id(k, j)]) % mod;
            }
        }
    }
    return c;
}

Matrix operator+(const Matrix &a, const Matrix &b) {
    Matrix c{};
    for (int i = 0; i < M * M; i++) {
        c[i] = (a[i] + b[i]) % mod;
    }
    return c;
}

Matrix power(Matrix a, i64 b) {
    Matrix ans{};
    init(ans, 1);
    for (; b; b >>= 1, a = a * a) {
        if (b & 1) ans = ans * a;
    }
    return ans;
}
```

1.13 哥德巴赫猜想

每个大于 2 的偶数都能表示成两个质数之和

每个大于 5 的奇数都能表示成三个质数之和

1.14 整除分块

我们考虑如何解决各式各样的整除分块

我们先考虑一维的情况

即 $\sum_{i=1}^n \lfloor \frac{m}{f(i)} \rfloor$ 其中 m 是常数 $f(i)$ 为单元函数且单调增加, 我们只需要每次求出右端点 R , 然后 $L = R + 1$ 即可, 问题在于如何求出 R , 我们来推式子

首先我们已经知道左端点 L , 也就相当于知道了当前的值 val 也就是说 $\lfloor \frac{m}{f(L)} \rfloor = val$ 那么 R 应该满足 $R = \max(i)$ 并且 $i * val \leq m$, 那么 $R = \lfloor \frac{m}{val} \rfloor = \lfloor \frac{m}{\lfloor \frac{m}{f(L)} \rfloor} \rfloor$

然后我们考虑二维的情况, 一般形式为 $\sum_{i=1}^{\min(n, m)} \lfloor \frac{n}{i} \rfloor \lfloor \frac{m}{i} \rfloor$, 在这种情况下我们发现有两个限制, 但我们只需要每次操作取他们的交集即可, 因为有 $L = R + 1$ 在。可以保证全覆盖

也就是取 $R = \min(\lfloor \frac{n}{\lfloor \frac{n}{i} \rfloor} \rfloor, \lfloor \frac{m}{\lfloor \frac{m}{i} \rfloor} \rfloor)$, 至于函数的形式我们参考一维即可

```
std::vector<std::pair<ll, ll>> floorBlock(ll n, ll m)
// n/i i 从 1 到 m 的整除分块区间
{
    std::vector<std::pair<ll, ll>> res;
    for (ll l = 1, r; l <= std::min(n, m); l = r + 1)
    {
        r = std::min(n / (n / l), std::min(n, m));
        res.emplace_back(l, r);
    }
    return res;
}
```

上取整的整除分块

```
std::vector<std::pair<ll, ll>> ceilBlock(ll n)
// n/i 向上取整的分块区间
{
    std::vector<std::pair<ll, ll>> res;
    for (ll l = 1, r; l <= n; l = r + 1)
    {
        ll val = (n + l - 1) / l;
        r = (val == 1 ? n : (n - 1) / (val - 1));
        res.emplace(l, r);
    }
    return res;
}
```